

Monitoramento e otimização de recursos em aplicativos Flutter

Beatriz Vocurca Frade

Universidade Federal de Minas Gerais
Engenharia de Sistemas
Orientador: Prof. Ramon Lacerda Marques
2026



Visão geral da defesa

Mapa da apresentação

1. Problema

Introdução, contexto Flutter, motivação e objetivos.

2. Base científica

Trabalhos relacionados e lacuna de pesquisa.

3. Humanidades

Sustentabilidade digital, inclusão, ética e privacidade.

4. Solução

Arquitetura, módulos, métricas e dashboard.

5. Método

Requisitos, experimentos e cenários de teste.

6. Fechamento

Resultados, contribuições, limitações e conclusão.

Introdução

Capítulo 1 – Introdução

- Flutter consolidou-se como um dos principais caminhos para apps multiplataforma.
- Interfaces ricas aumentam pressão sobre CPU, GPU, memória e rede.
- Em dispositivos móveis, pequenas ineficiências viram experiência ruim: travamentos, quedas de FPS e drenagem de bateria.
- O diagnóstico de desempenho precisa acontecer durante o desenvolvimento, não apenas no fim do projeto.

Ideia central

Performance é uma conversa contínua com o aplicativo: cada frame conta um pouco da história.

Termômetro embarcado

Problema de pesquisa

Capítulo 1 – Problema

Ferramentas atuais

Flutter DevTools, *Android Profiler*, *Perfetto* e *Xcode Instruments* oferecem dados ricos, mas exigem troca de contexto, configuração manual e interpretação especializada.

Consequência prática

O diagnóstico tende a ser adiado. Quando a regressão aparece, ela já pode ter chegado ao usuário.

Como tornar o diagnóstico de performance mais acessível, contínuo e acionável em Flutter?

Motivação

Capítulo 1 – Motivação e justificativa

Experiência

jank, lentidão e memória alta tornam o app menos confiável.

Energia

Renderização, rede e CPU elevam consumo de bateria e aquecimento.

Inclusão

Apps leves funcionam melhor em aparelhos antigos ou de entrada.

- A literatura relaciona métricas observáveis de execução ao consumo energético.
- Em Flutter, *rebuilds* frequentes e frames longos são sintomas importantes.
- A lacuna está entre “ter dados” e “saber agir sobre eles”.

Objetivos

Capítulo 1 – Objetivos

Objetivo geral

Desenvolver e avaliar o `collector_flutter`, uma biblioteca Dart/Flutter embarcada para coletar métricas, analisar padrões de degradação e recomendar otimizações em tempo real.

- Coletar FPS, *jank*, memória, rede, eventos e *rebuilds*.
- Gerar recomendações heurísticas por severidade.
- Projetar arquitetura modular e extensível.
- Manter *overhead* alvo inferior a 5%.
- Exibir dados em um dashboard integrado ao app.
- Apoiar ensino, pesquisa e reprodutibilidade.

Trabalhos relacionados I: energia e métricas

Capítulo 2 – Revisão bibliográfica

Autor	Contribuição	Relação com este trabalho
Pathak et al. (2011)	Modelagem energética fina por chamadas de sistema em smartphones Android.	Mostra que rede e interface pesam no consumo energético.
Hao et al. (2013)	Estimativa de consumo energético por análise de programa.	Fundamenta proxies de energia sem instrumentação física dedicada.
Li et al. (2014)	Estudo empírico de energia em apps Android.	Reforça a medição por métricas observáveis de execução.
Hoque et al. (2015)	Revisão sobre modelagem, profiling e depuração de energia em dispositivos móveis.	Organiza métodos, limites e sinais úteis para diagnóstico.

Síntese: energia pode ser inferida por sinais como CPU, frames, memória e rede.

Trabalhos relacionados II: profiling e Flutter

Capítulo 2 – Revisão bibliográfica

Autor	Contribuição	Relação com este trabalho
Ravindranath et al. (2012)	Monitoramento de desempenho móvel em aplicações reais com AppInsight.	Inspira coleta próxima do contexto real de execução.
Rua e Saraiva (2024)	Estudo empírico em larga escala sobre energia, tempo de execução e memória.	Justifica analisar múltiplos sinais em conjunto.
Nawrocki et al. (2021)	Comparação entre frameworks nativos e multiplataforma.	Situa Flutter no debate de desempenho cross-platform.
Flutter DevTools (2026)	Ferramentas oficiais para inspecionar frames, memória e execução em Flutter.	Alinha as métricas do protótipo ao ecossistema Flutter.

Síntese: em Flutter, renderização e reconstrução de widgets são pontos críticos.

Trabalhos relacionados III: automação e recomendações

Capítulo 2 – Estudos acadêmicos relacionados

Autor	Contribuição	Relação com este trabalho
Sun et al. (2023)	Revisão sobre diagnóstico de ineficiências energéticas em apps Android.	Ajuda a transformar sintomas em hipóteses de causa.
Bangash et al. (2023)	Estimativa de energia a partir do uso de APIs em apps móveis.	Apoia heurísticas baseadas em eventos e padrões de uso.
Android Profiler e Perfetto	Profiling detalhado de CPU, memória, rede, energia e tracing.	Servem como referência para comparação técnica.
Instruments e Firebase	Profiling no ecossistema Apple e monitoramento de performance em produção.	Mostram forças e limites das soluções externas.
<code>collector_flutter</code>	Monitoramento embarcado com recomendações acionáveis.	Atua na lacuna entre medir e decidir o próximo passo.

Lacuna de pesquisa

Capítulo 2 – Síntese crítica

O que já existe

- Métricas maduras de profiling.
- Ferramentas oficiais detalhadas.
- Estudos sobre energia, memória, rede e renderização.

O que falta

- Coleta dentro do app monitorado.
- Baixa fricção de uso no ciclo diário.
- Recomendações automáticas e acionáveis.

O `collector_flutter` ocupa essa lacuna: menos laboratório externo, mais diagnóstico no fluxo real de desenvolvimento.

Contextualização em humanidades

Capítulo 3 – Contexto social, ambiental e ético

Sustentabilidade digital

Software ineficiente consome mais energia, aquece o dispositivo e pode acelerar obsolescência percebida.

Ética e privacidade

O protótipo coleta métricas técnicas de desempenho, sem dados pessoais ou conteúdo do usuário.

Inclusão digital

Otimizar performance amplia a chance de o app funcionar bem em dispositivos de entrada.

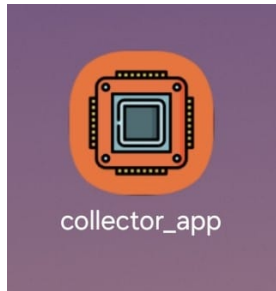
Responsabilidade

Diagnosticar consumo de recursos é parte da qualidade social e ambiental do software.

Proposta

Capítulo 4 – Descrição do protótipo

- `collector_flutter` é um pacote Dart/Flutter de monitoramento embarcado.
- Funciona como uma tela do próprio app, sem servidor externo.
- Coleta métricas, aplica análise heurística e exibe recomendações.
- Foco: acessibilidade, reprodutibilidade e baixo acoplamento.



Em uma frase

Um painel de diagnóstico que mora dentro do aplicativo e transforma telemetria em ação.

Ferramentas atuais: comparação

Capítulo 2 – Ferramentas de monitoramento existentes

Ferramenta	Força principal	Limitação para o caso de uso
<i>Flutter DevTools</i>	Timeline, widgets, memória e CPU.	Exige ferramenta separada e interpretação manual.
Perfetto	Traços detalhados de sistema Android.	Baixo nível, configuração manual e alto custo cognitivo.
Android Profiler	CPU, memória, rede e energia no Android Studio.	Restrito a Android e dependente de ambiente externo.
Xcode Instruments	Profiling avançado para iOS/macOS.	Ecossistema Apple, coleta manual e curva de aprendizado.
Firebase Performance	Monitoramento em produção.	Pouca granularidade para renderização e CPU.
Flipper / LeakCanary	Depuração e memória em ecossistemas específicos.	Não cobrem o fluxo Flutter completo.

Arquitetura geral

Capítulo 4 – Arquitetura da solução

Data

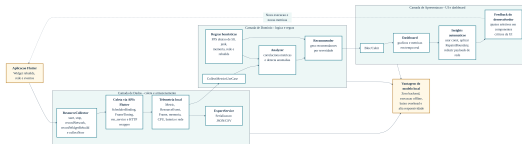
Fontes de dados para frames, memória, rede e eventos.

Domain

Entidades, casos de uso, Analyzer e Recommender.

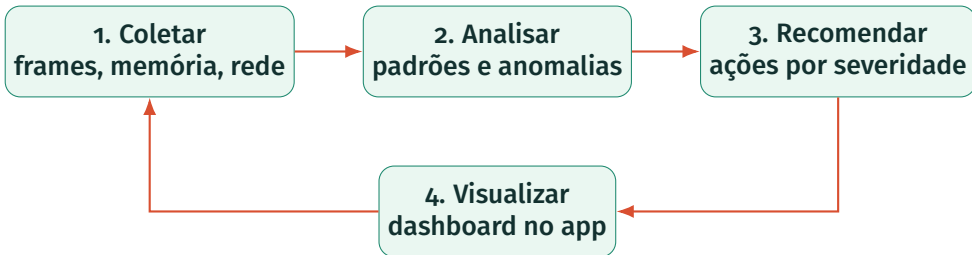
Presentation

CollectorBloc, DashboardPage e visualização reativa.



Fluxo do sistema

Capítulo 4 – Coleta, análise e recomendação

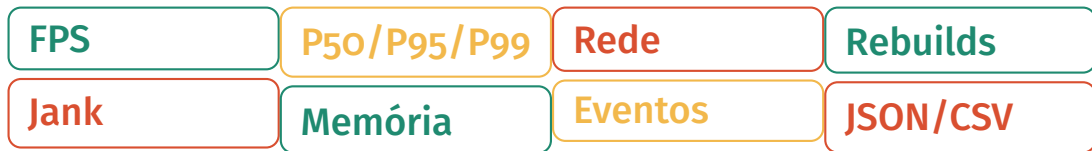


Leitura lúdica

O app envia sinais vitais; o coletor escuta, interpreta e devolve uma prescrição de otimização.

Métricas coletadas

Capítulo 4 – Instrumentação



- Frames: SchedulerBinding e FrameTiming.
- Memória: RSS e heap via VM Service, com fallback quando necessário.
- Rede: cliente HTTP instrumentado com URL, status, latência e bytes.

Módulos principais

Capítulo 4 – Componentes

Módulo	Papel na solução
ResourceCollector Collector / fontes de dados	Fachada pública: inicia, para, coleta agora e gerencia ciclo de vida. Capturam frames, memória, rede, eventos e <i>rebuilds</i> .
Analyzer	Resume métricas e detecta sinais de degradação.
Recommender	Converte sintomas em recomendações por severidade.
DashboardPage	Exibe cartões, gráficos, rede e recomendações dentro do app.
ExportService	Serializa a sessão em JSON e frames em CSV para análise externa.

Integração no app

Capítulo 4 – Uso prático

```
final collector = ResourceCollector(  
  collectionInterval: const Duration(seconds: 2),  
  budget: const PerformanceBudget(targetFrameRate: 60),  
);  
  
await collector.start();  
  
Navigator.push(  
  context,  
  MaterialPageRoute(  
    builder: (_) => DashboardPage(collector: collector),  
  ),  
);  
  
await collector.network.get(Uri.parse('https://api.exemplo.com/dados'));  
collector.recordEvent('checkout_started', {'items': 3});
```

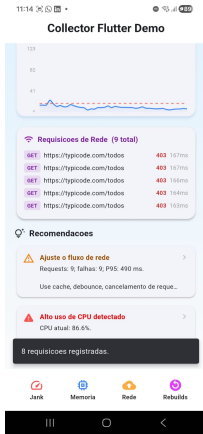
A proposta é reduzir fricção: poucas chamadas ativam coleta, painel e instrumentação de rede.

Protótipo visual

Capítulo 4 – Dashboard integrado



Estado saudável

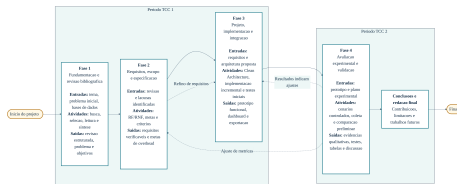


Gargalos detectados

Metodologia

Capítulo 4 – Metodologia e desenvolvimento

- Pesquisa aplicada: constrói uma solução prática para um problema específico.
- Abordagem experimental: observa variáveis de desempenho sob cenários controlados.
- Ciclo iterativo e incremental: revisão, requisitos, implementação e validação.
- Rastreabilidade entre requisitos, módulos e critérios de avaliação.



Requisitos do sistema

Capítulo 4 – Requisitos funcionais e não funcionais

Funcionais

- Coletar métricas em tempo real.
- Correlacionar degradações.
- Gerar recomendações.
- Exibir dashboard integrado.
- Simular cenários de carga.

Não funcionais

- Operar integralmente em Dart.
- *overhead* alvo abaixo de 5%.
- Interface fluida e responsiva.
- Modularidade e separação de responsabilidades.
- Extensibilidade para novas métricas.

Procedimentos de teste

Capítulo 4 – Procedimentos experimentais

- Execução preferencial em modo *profile*.
- Preparação do ambiente antes de cada execução.
- Cenários com duração controlada e repetição.
- Coleta estruturada em JSON e CSV.
- Comparação planejada com Android Profiler e `adb shell dumpsys gfxinfo`.
- Cálculo de erro percentual para precisão.
- Cálculo de *overhead* sobre tempo médio de frame.
- Análise estatística para validação ampliada.

Critério científico

Resultados quantitativos entram apenas quando sustentados por protocolo e repetição suficientes.

Cenários experimentais

Capítulo 4 – Cargas controladas

1. Rebuilds	2. Rede	3. Memória	4. Combinado
Reconstruções intensivas de interface.	Requisições HTTP simultâneas.	Alocação e liberação de objetos grandes.	Interface, rede e memória ao mesmo tempo.

Cada cenário provoca um sintoma; o dashboard precisa perceber e explicar.

Resultados implementados

Capítulo 5 – Resultados e discussão

- Pacote `collector_flutter` estruturado como biblioteca Flutter plugin.
- Coleta de frames, memória, rede, eventos customizados e *rebuilds*.
- Coleta de CPU via *platform channel* Android (`/proc/stat`).
- Monitoramento de bateria via `BatteryManager` (Android) e `UIDevice` (iOS).
- Dashboard com cartões de CPU, bateria e demais métricas.
- Exportação JSON e CSV via `ExportService`.
- 48 testes automatizados cobrindo `Analyzer`, `Recommender`, `ExportService`, `CpuDataSource` e `BatteryDataSource`.
- Heurísticas de CPU (alerta acima de 60% e 80%) e bateria (alerta abaixo de 20%).
- Recomendações de `Isolate` para CPU alta e redução de coleta para bateria baixa.
- Correções de sincronização, gráfico, URL e histórico de memória.

Resultados qualitativos

Capítulo 5 – Validação preliminar

Cenário	Comportamento observado
Rebuilds intensivos	Quedas de FPS e frames longos foram detectados; recomendações sugeriram <code>const</code> , isolamento de reconstruções e padrões reativos.
Rede simultânea	Requisições foram registradas com URL, status e latência; alto volume gerou alerta de cache/debounce.
Memória intensiva	Crescimento de RSS foi acompanhado com suavização para evitar falsos positivos transitórios.
Carga combinada	Múltiplas métricas foram processadas sem interferência entre módulos; recomendações foram priorizadas por severidade.

Esses resultados sustentam a operacionalidade do protótipo; a validação quantitativa ampla é uma evolução natural.

Evolução do protótipo

Capítulo 5 – Limitações superadas

Limitações superadas

- **CPU:** *platform channel* Android (/proc/stat).
- **Bateria:** BatteryManager (Android) e UIDevice (iOS).
- **Exportação:** JSON e CSV via ExportService.
- **Testes:** 48 testes unitários automatizados.

Trabalhos futuros

- CPU completo no iOS (Mach APIs).
- Experimentos quantitativos formais (≥ 30 repetições, 3 dispositivos).
- Comparação estatística com ferramentas de referência.
- Avaliação de usabilidade (ISO 9241-11).

Contribuições

Capítulo 6 – Conclusões

Técnica

Biblioteca aberta, modular e embarcada para diagnóstico de performance em Flutter, disponível no GitHub e no pub.dev.

Científica

Arquitetura, requisitos, cenários e protocolo orientados à reprodutibilidade.

Social

Apoio a software mais eficiente, inclusivo e consciente de privacidade.

Monitorar melhor é uma forma de projetar melhor.

Conclusão

Capítulo 6 – Considerações finais

- O trabalho mostrou que é viável aproximar o diagnóstico de performance do fluxo cotidiano de desenvolvimento Flutter.
- O `collector_flutter` evoluiu de 4 para 6 métricas (CPU e bateria), com 48 testes e exportação integrada.
- A arquitetura em camadas favoreceu extensão e testabilidade: os novos módulos foram adicionados sem alterações nas camadas superiores.
- A contribuição extrapola desempenho: eficiência também dialoga com sustentabilidade, inclusão digital e responsabilidade tecnológica.

Mensagem final

Performance não é apenas fazer o app correr mais rápido; é fazer o app respeitar melhor o dispositivo, a bateria e a pessoa que o usa.

Perguntas?

collector_flutter

UF *m* G



Beatriz Vocurca Frade | Engenharia de Sistemas – UFMG